

Week 7: LLMs (large language models)

Agenda

Any questions left over from last week (or from whenever)?	1
Just another word on scale	1
Large Language Models Architecture	1
Resources to watch	3
Looking at some code	3

Any questions left over from last week (or from whenever)?

Just another word on scale

c.f. LAION 2b or 5b: more data makes the model *worse*. Why?

- more difficult to keep larger datasets clean
- but also, the distribution of everything $\neq$ the distribution you want

GPT-3: 175b parameters (as opposed to 13k)

OpenAI's drive to scale

→ the scale of a LLM matters very much, too

Large Language Models Architecture

What are Large Language Models?

Short answer = they're language creating generative models. Eg. ChatGPT, Claude, etc.

The basic idea = predicting the next most likely word

Let's dive into this a little bit. When you play a game of chance with dice, and you want to win, you would want to maximize your ability to get the next roll of the dice right. What is the likelihood that the next roll will be a 6?

→ for all possible outcomes (1, 2, 3, ..., 6) the probability is 1/6

This is called a probability distribution. If I know the probability distribution of all the possible outcomes of a roll of the dice, then I have the best chance of winning the game.

With words, the challenge is: given the user's prompt, what's the next most likely word? And given that word, what's the next most likely word after that?

→ To perform well at this, what we would want is to learn the probabilities of all possible outcomes (= words)

In other words, we want to learn the probability distribution of all words.

HOWEVER, there are two important differences:

- First, we're not really learning the probabilities of individual words, but of so-called **tokens** = sequences of letters, such as *th*, *ing*, *ere*, and so on. But I'll continue talking as if we were learning about words, because it's more intuitive.
 - Second, **context matters**: a given word (say "fixture") doesn't have the same probability no matter what, because it doesn't have the same meaning in all contexts. For example, if a word has just occurred, it's unlikely to occur immediately again. Or, "fixture" could have a higher probability in contexts where Manchester United and light fittings are being discussed, but lower probabilities when you're discussing fixing a bike. LLMs have a special architecture designed to deal with this.
- ⇒ In sum, LLMs generate text by predicting the next best word given the prompt. They do this by learning about the probability distribution of all possible words ("words") given their context.
- ⇒ And the way they deal with context has to do with so-called **embedding** and so-called **attention**.

The two-step process of training

1. Pre-training (GPT)
2. Reinforcement learning with human feedback (RLHF)

But the core innovation is the model's architecture, and it's known as a **transformer** (GPT):

Transformer = a type of neural network, consists of

- **Embedding matrix** (vectors to "place words in space") – cf notes on the intuition of capturing words in vector space
- **Attention** – embeddings are first passed through this, and updated (context size)—creates a context-aware representation of each word/token
- **Feedforward neural network (MLP = multi-layer perceptron)** – then embeddings are passed through this—its purpose is to transform previous layers' outputs into easily workable formats
- (there are some normalization steps in between)
- The last operation (Softmax) produces a probability distribution for all possible tokens

Please watch some of the 3Blue1Brown videos on these topics for an intuitive understanding of these layers;

You can also read about them in plain English here:

<https://medium.com/@abdullah.afify/a-detailed-simplified-explanation-of-the-transformers-architecture-125c3b33b6cb>

Resources to watch

3Blue1Brown [Large Language Models explained briefly](#) – very good intuitive explanation of LLMs by applying the intuition we’ve seen on neural networks thus far. **Very good, watch this.**

[Transformers, the tech behind LLMs / Inside an LLM](#) – gives a good, if somewhat too detailed, explanation of what all the different parts of the Transformer architecture do, including an explanation of embeddings. **Very good, watch this.**

[Attention in Transformers, step-by-step](#) – this video can be helpful, but it does focus a bit heavily on the math, specifically on how different matrices are formed and multiplied with one another in the attention process. It’s a bit beyond the scope of our module. / The resources at the end are interesting though!

[How might LLMs store facts](#) – this video is quite good again: it really helps embed some insights on how information is captured, which we would be familiar with by this time in the module, but does so with LLMs in mind, adding some aspects of how LLMs work and helping understanding for LLMs as well.

Visual explanations of transformers:

<https://poloclub.github.io/transformer-explainer/>
<https://transformer.nexgencompany.ai/>

Looking at some code

Just be aware that this is A LOT, so we’ll be a bit cursory...

This is some code for using this architecture to train a Small language model. As you’ll see, it’s not that small...

The link to the Github where the code comes from is here:

<https://github.com/ChaitanyaK77/Building-a-Small-Language-Model-SLM-/blob/main/README.md> and all credit goes to Chaitanya Kakade. (But also, sorry on his behalf for typos and a bit of messiness in the code.)

What we’re doing here is *pre-training only*. (Consider that in terms of resources...)

The code has the following parts:

1. Load and prepare dataset, including tokenizing, prepping batches and reserving GPU memory if relevant
DEFINE ARCHITECTURE
2. Create classes using nn.Module for the different components of the model architecture, including:
 - a. Layer normalization (= normalization layer)
 - b. Self-attention (= attention)
 - c. MLP
 - d. The above (layernorm, attention, MLP) are then integrated into the Transformer Block (called Block)
3. After that, GPTConfig creates a configuration variable with hyperparameters
4. and the **GPT class is defined for the model itself**, which consists of multiple embedding matrices, transformers, etc. *This is complicated.*
5. In the same cell block, we have the definition of the generation process.
PREPARE TRAINING
6. Then we set the Config variable and define the model instance.
7. Define the loss function (cross entropy loss)
8. More training configuration things (two cell blocks)

Took me 489m (=almost exactly 6 hours to train). GPT-3 took OpenAI months on a supercomputer, with maybe 10k GPUs...

Stochastic Parrots paper: <https://dl-acm-org.salford.idm.oclc.org/doi/10.1145/3442188.3445922>